Handout 1 · Vocabulary

Simulation and physics

Term	Meaning
Simulation	Running a model of a robot and its world on a computer to predict what would happen in reality.
Physics engine	Software that computes motion from forces: gravity, contact, friction, motors. MuJoCo, PhysX and Newton are physics engines.
Time step	The small slice of time the engine advances per update (2 ms in the demos). Too large and contacts become unstable.
Real-time factor	Simulated seconds per wall-clock second. 100× means 100 s of robot experience every second.
Rigid body	An object assumed not to bend. Most robot links are modelled this way.
Joint	A connection that lets two bodies move relative to each other: hinge (rotates), slide (translates), free (6 degrees of freedom).
Degrees of freedom (DoF)	The number of independent ways something can move. A free-floating body has 6.
Contact model	How the engine decides that two objects touch and what forces they exchange.
Friction coefficient	A number describing how strongly surfaces resist sliding. Steel on a smooth floor is low; rubber is high.
Restitution	Bounciness: how much speed survives a collision.
Collision geometry	Simple shapes (boxes, spheres, capsules) used for physics, often separate from the detailed visual mesh.
Actuator	Anything that makes a robot move: a motor, a servo, a piston.
Servo	A motor with a built-in controller that holds a target position or velocity.

Describing robots and scenes

Term	Meaning
MJCF	MuJoCo's XML format describing bodies, joints, actuators and sensors. `demo/assets/scad_lab.xml` is MJCF.
URDF	Unified Robot Description Format; the ROS standard XML for robot links and joints.
OpenUSD	Universal Scene Description; a scene format from Pixar's film pipeline, used by Isaac Sim and Omniverse.
Digital twin	A simulation kept closely matched to a specific real robot, room or factory.
ROS 2	Robot Operating System 2; the common middleware that connects sensors, algorithms and motors on real robots and in simulators.

Sensing and perception

Term	Meaning
Sensor emulation	Producing the data a real sensor would produce, including its range, rate and noise.
Encoder	Measures how far or fast a wheel or joint has turned.
IMU	Inertial Measurement Unit: an accelerometer and a gyroscope measuring acceleration and rotation rate.
Lidar	A laser range sensor that measures distances along many beams; the demo rover has 15.
RGB-D camera	A camera that returns colour (RGB) and depth (D) for every pixel.
Ground truth	The exact, true state of the world, known in simulation and only estimated on hardware.
Perception pipeline	The chain of steps that turns raw sensor data into decisions, e.g. image → threshold → blob → bearing → steering.
Occupancy grid	A map divided into cells, each marked free or occupied; built from lidar in Demo 3.
SLAM	Simultaneous Localization and Mapping: building a map while working out where you are in it.
Synthetic data	Simulated images or scans with automatic labels, used to train perception models.
Sensor fidelity	How closely a simulated sensor matches the real one. Ray-traced sensors are higher fidelity than idealised ones.

Control and autonomy

Term	Meaning
Differential drive	Steering by running left and right wheels at different speeds.
Kinematics	The geometry of motion, without forces: how wheel speeds become robot motion.
Open-loop control	Sending pre-planned commands without checking the result.
Closed-loop (feedback) control	Measuring the result and correcting continuously.
Proportional (P) controller	A feedback controller whose correction is proportional to the error.
Path planning (A*)	Searching a map for the shortest collision-free route.
Pure pursuit	Following a path by steering toward a point a fixed distance ahead on it.
Sense–plan–act loop	The cycle at the heart of robot autonomy: read sensors, decide, move, repeat.

Sim-to-real and scale

Term	Meaning
Reality gap / sim-to-real gap	The difference between simulated and real behaviour.
Sim-to-real transfer	Moving software developed in simulation onto a physical robot.
Domain randomization	Varying physics, visuals and sensors across many simulated worlds so software works in all of them — including the real one.
System identification	Measuring a real robot's properties to tune the simulator to match.
Latency	Delay between sensing and acting.
Hardware-in-the-loop	Testing real hardware components connected to a simulation.

Term	Meaning
GPU acceleration	Running physics (and rendering) on a graphics card to simulate thousands of worlds in parallel.
Reinforcement learning (RL)	Learning a controller by trial and error from rewards, usually inside a simulator.
Parallel environments	Many copies of a simulation stepped at once, for testing or training.

github.com/mrpillis/scad-robotics-simulation-demo

Handout 2 · Additional Learning Resources

Links checked October 2026. ★ = good first step.

Start today

- ★ **This repository** — every demo runs on a laptop: `pip install -r requirements-demo.txt`, then `python demo/02_drive_the_robot.py`.
- ★ **MuJoCo tutorial notebook** (runs in the browser on Google Colab) — https://colab.research.google.com/github/google-deepmind/mujoco/blob/main/python/tutorial.ipynb
- **MuJoCo Playground notebooks** — GPU robot learning in Colab: locomotion, manipulation, vision — https://playground.mujoco.org
- **mjlab demo, no install** — `uvx --from mjlab --refresh demo` — https://github.com/mujocolab/mjlab
- **ArtifactArena Play** — design a robot as MJCF + Python and test it in the browser — https://artifactarena.ai/play

Platform documentation

Platform	Link
MuJoCo	https://mujoco.readthedocs.io
MuJoCo Warp	https://github.com/google-deepmind/mujoco_warp
mjlab	https://github.com/mujocolab/mjlab
NVIDIA Isaac Sim	https://docs.isaacsim.omniverse.nvidia.com
NVIDIA Isaac Lab	https://isaac-sim.github.io/IsaacLab
Unity Robotics Hub	https://github.com/Unity-Technologies/Unity-Robotics-Hub
Unity ML-Agents	https://github.com/Unity-Technologies/ml-agents
Gazebo	https://gazebosim.org
Webots	https://cyberbotics.com
Newton	https://github.com/newton-physics/newton
ROS 2	https://docs.ros.org
OpenUSD	https://openusd.org

Books (several free online)

- ★ Kevin Lynch and Frank Park, *Modern Robotics: Mechanics, Planning, and Control* (Cambridge, 2017). Free preprint and videos: https://hades.mech.northwestern.edu/index.php/Modern_Robotics
- Roland Siegwart, Illah Nourbakhsh and Davide Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. (MIT Press, 2011). Mobile robots, sensors, mapping — the topics of Demos 2–4.
- Sebastian Thrun, Wolfram Burgard and Dieter Fox, *Probabilistic Robotics* (MIT Press, 2005). The reference for mapping, localisation and SLAM.
- Steven LaValle, *Planning Algorithms* (Cambridge, 2006). Free: http://lavalle.pl/planning/

- Russ Tedrake, *Underactuated Robotics* and *Robotic Manipulation* (MIT course notes, free, with runnable notebooks): https://underactuated.mit.edu · https://manipulation.mit.edu
- Kevin Lynch, Nicholas Marchuk and Matthew Elwin, *Embedded Computing and Mechatronics with the PIC32 Microcontroller* (Newnes, 2015), for students moving from simulation to their own hardware.

Video courses and lectures

- *Modern Robotics* specialization (Northwestern, Coursera) — video lectures that accompany the book; linked from the book page above.
- Cyrill Stachniss, lectures on mobile robotics, SLAM and photogrammetry (University of Bonn), free on YouTube — search "Cyrill Stachniss SLAM course".
- MIT *Robotic Manipulation* and *Underactuated Robotics* lecture videos (Russ Tedrake) — linked from the course sites above.
- NVIDIA Deep Learning Institute — free self-paced Isaac Sim and Isaac Lab courses: https://learn.nvidia.com

Papers behind the ideas in the demos

- Todorov, Erez, Tassa, "MuJoCo: A physics engine for model-based control," IROS 2012.
- Tobin et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," IROS 2017. https://arxiv.org/abs/1703.06907
- Zhao, Queralta, Westerlund, "Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey," 2020. https://arxiv.org/abs/2009.13303
- Rudin et al., "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning," CoRL 2021. https://arxiv.org/abs/2109.11978
- Zakka et al., "mjlab: A Lightweight Framework for GPU-Accelerated Robot Learning," 2026. https://arxiv.org/abs/2601.22074
- Tiwary, Mayo et al., "ArtifactArena: Evaluating Models by What They Build in the Physical World," 2026. https://artifactarena.ai/paper

Communities

- MuJoCo discussions — https://github.com/google-deepmind/mujoco/discussions
- ROS Discourse (ROS 2 and Gazebo) — https://discourse.openrobotics.org
- NVIDIA Isaac forums — https://forums.developer.nvidia.com

Handout 3 · Leave-Behind Project: "Teach a Robot to Find Its Way"

Something to continue your learning after today. Everything runs on a laptop. Choose **one track** at the end; Parts 1–3 are common to everyone.

```
git clone https://github.com/mrpillis/scad-robotics-simulation-demo && cd scad-robotics-simulation-demo
pip install -r requirements-demo.txt
python demo/03_sensor_dashboard.py
```

Part 1 · Rebuild the room (about 1 hour)

Open `demo/assets/scad_lab.xml`.

1. Move two obstacles and add one of your own (a `box` or `cylinder` geom inside `<worldbody>`).
2. Change the floor's friction and the colours.
3. Run Demo 3 and press **g** for new goals. Does the robot still find its way?

Deliverable: a screenshot of your room and one sentence on what changed.

Part 2 · Change the robot (about 1–2 hours)

1. Make the wheels 50 % larger. Update `WHEEL_RADIUS` in `demo/scad_sim.py` to match. What happens to top speed? To turning?
2. Remove five of the fifteen lidar beams (sites and sensors). Where does the robot start to fail?
3. Move the camera higher on the mast. What changes in Demo 4?

Deliverable: a short table: change → what you expected → what actually happened.

Part 3 · Break it on purpose (about 1 hour)

Use `demo/05_sim_to_real_gap.py`.

1. Run with `--strength 2`. Which worlds fail, and why? (Look at the printed friction and motor values.)
2. In `scad_sim.Rover`, raise `latency_steps` until closed-loop control starts to fail. How many milliseconds is that?

Deliverable: one paragraph: what is the most important difference between your simulation and a real robot, and how would you test it?

Part 4 · Choose a track (1–3 weeks)

Track A · Studio (art, design, film, games)

Design an environment with a narrative — a gallery, a set, a public space — and a robot whose path through it is part of the piece. Use the `studio` and `chase` cameras (or add your own) to record a 30-second sequence. Optionally rebuild the scene in Unity or Isaac Sim and compare the look and the behaviour.

Track B · Engineering (control and perception)

Replace the orange-threshold detector in Demo 4 with something more robust: detect a printed marker, or train a small classifier on images you render yourself with automatic labels. Measure success across lighting changes.

Track C · Research (learning and evaluation)

Pick one research question and answer it with experiments in simulation, e.g. - How does lidar beam count trade off against success rate and path length? - Can a learned policy (mjlab or MuJoCo Playground) match the A* planner here? - Design a robot for the ArtifactArena ring (Demo 7) by hand, with an AI assistant, or both, and compare.

Deliverable for all tracks: a 3-minute video or presentation, your modified files, and a one-page reflection: what simulation made possible, and what you would need to test on real hardware.

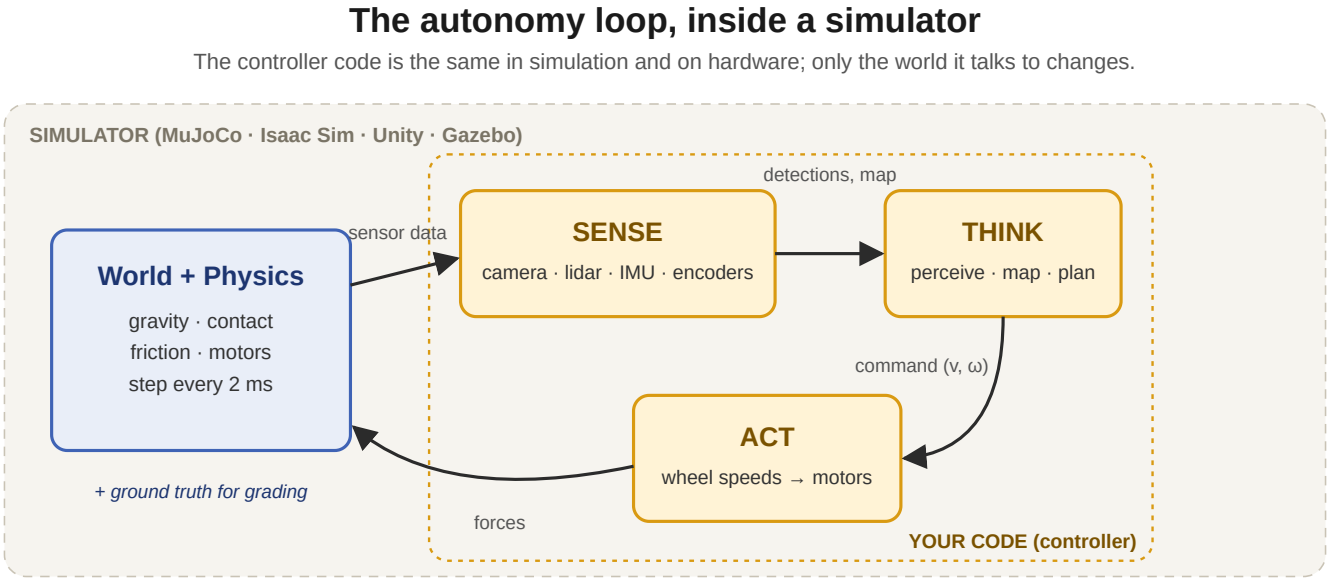
Assessment

Criterion	Excellent	Developing
Process	Changes are deliberate, documented and versioned	Changes are made but not explained
Understanding	Explains *why* the robot behaved as it did, using the vocabulary	Describes *what* happened only
Sim-to-real thinking	Names specific gaps and proposes a test for each	Mentions that reality is different
Communication	Clear visuals, readable files, concise reflection	Hard to follow or reproduce
Ambition	Goes beyond the starting code in a direction of your own	Stays within the examples

github.com/mrpillis/scad-robotics-simulation-demo

Handout 4 · Visual Guide

1 · The autonomy loop



Everything in the yellow box is your code. In simulation the arrows connect to a physics engine; on a robot they connect to drivers. Keep that boundary clean and the same code runs in both.

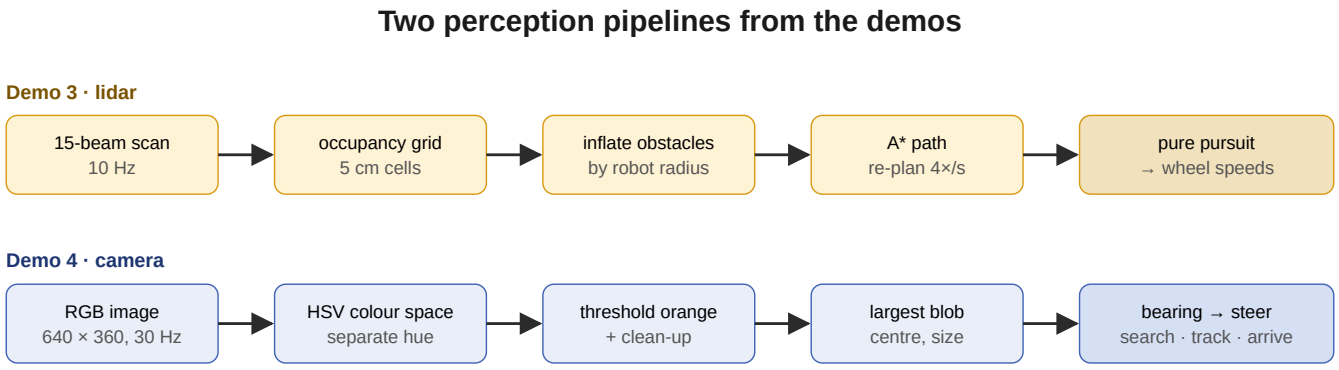
2 · Anatomy of a robot file

The demo rover, abridged from `demo/assets/scad_lab.xml`:

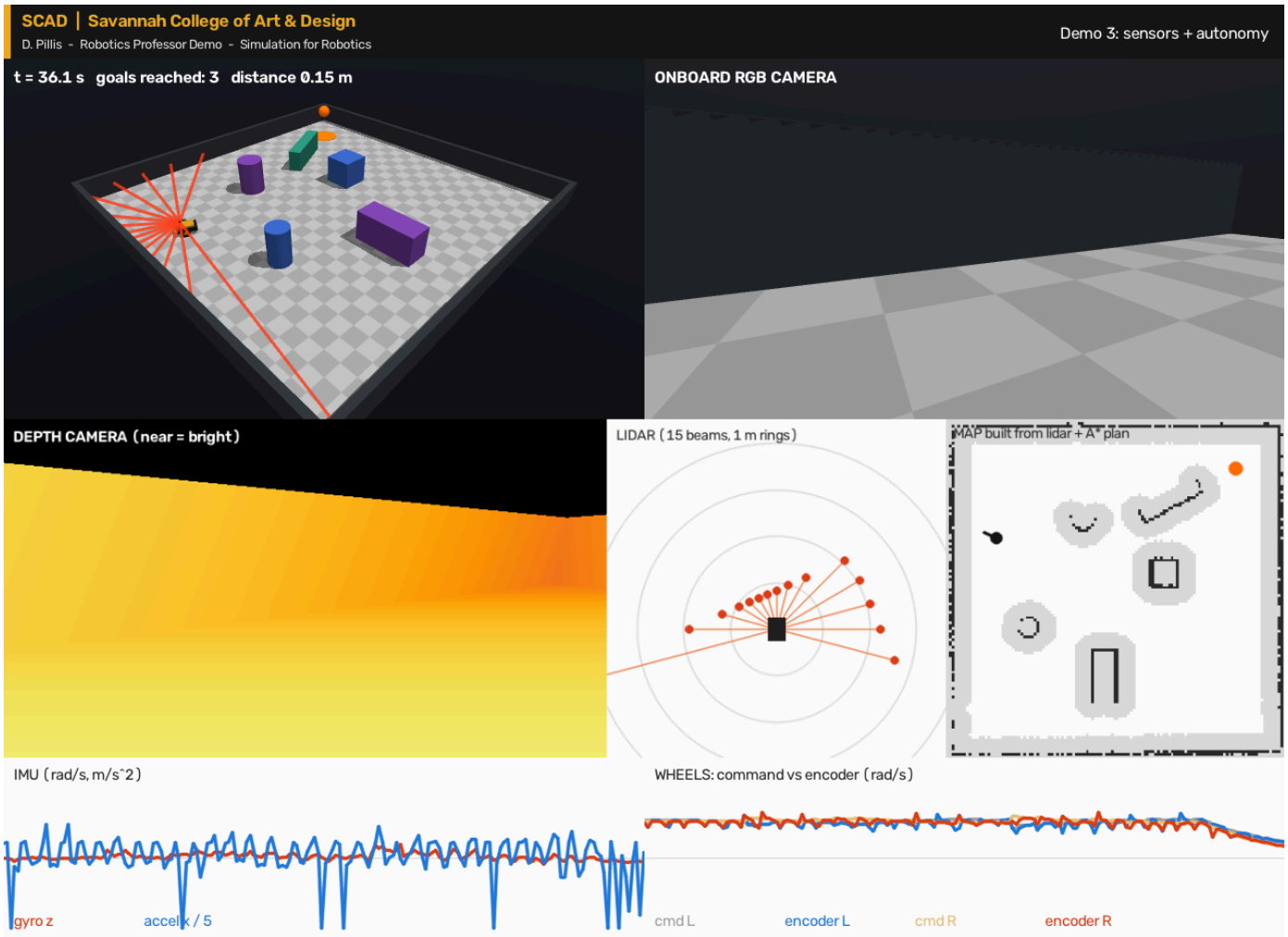
```
<body name="rover" pos="-2.2 -2.2 0.07">           <!-- WHERE: start pose -->
  <freejoint/>                                      <!-- HOW IT MOVES: free in 3-D -->
  <geom type="box" size="0.16 0.11 0.035" mass="3"/> <!-- WHAT: chassis shape and mass -->
  <camera name="rover_cam" pos="0.09 0 0.16" .../>   <!-- SENSE: onboard camera -->
  <site name="beam_07" zaxis="1 0 0"/>               <!-- SENSE: one lidar beam -->
  <body name="wheel_left" pos="0.03 0.135 0">
    <joint name="wheel_left" type="hinge" axis="0 1 0"/>
    <geom type="cylinder" size="0.07 0.02" friction="1.2"/>
  </body>
</body>

<actuator>
  <velocity name="motor_left" joint="wheel_left" kv="8" forcerange="-0.6 0.6"/>  <!-- ACT -->
</actuator>
<sensor>
  <jointvel name="enc_left" joint="wheel_left"/>     <!-- encoder -->
  <gyro name="imu_gyro" site="imu"/>                 <!-- IMU -->
  <rangefinder name="lidar_07" site="beam_07"/>      <!-- lidar beam -->
</sensor>
```

3 · Two perception pipelines

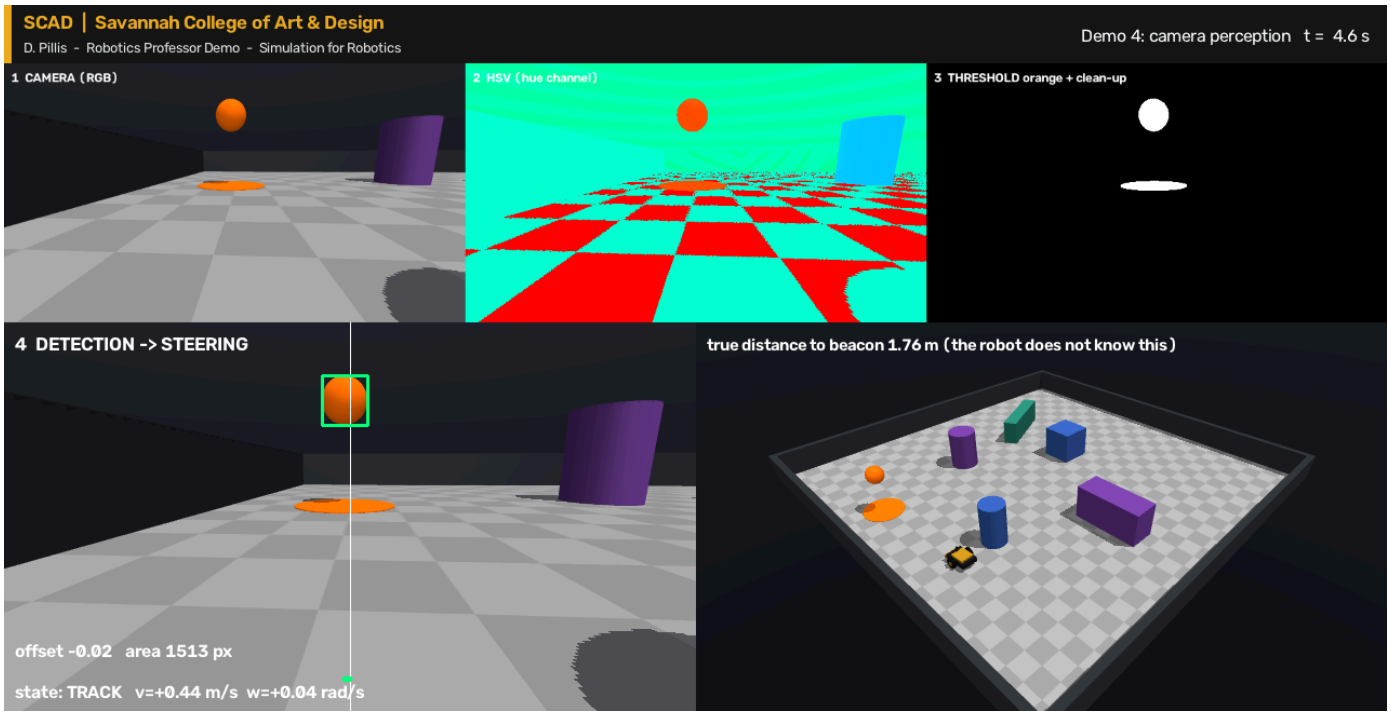


4 · One dashboard, every sensor (Demo 3)

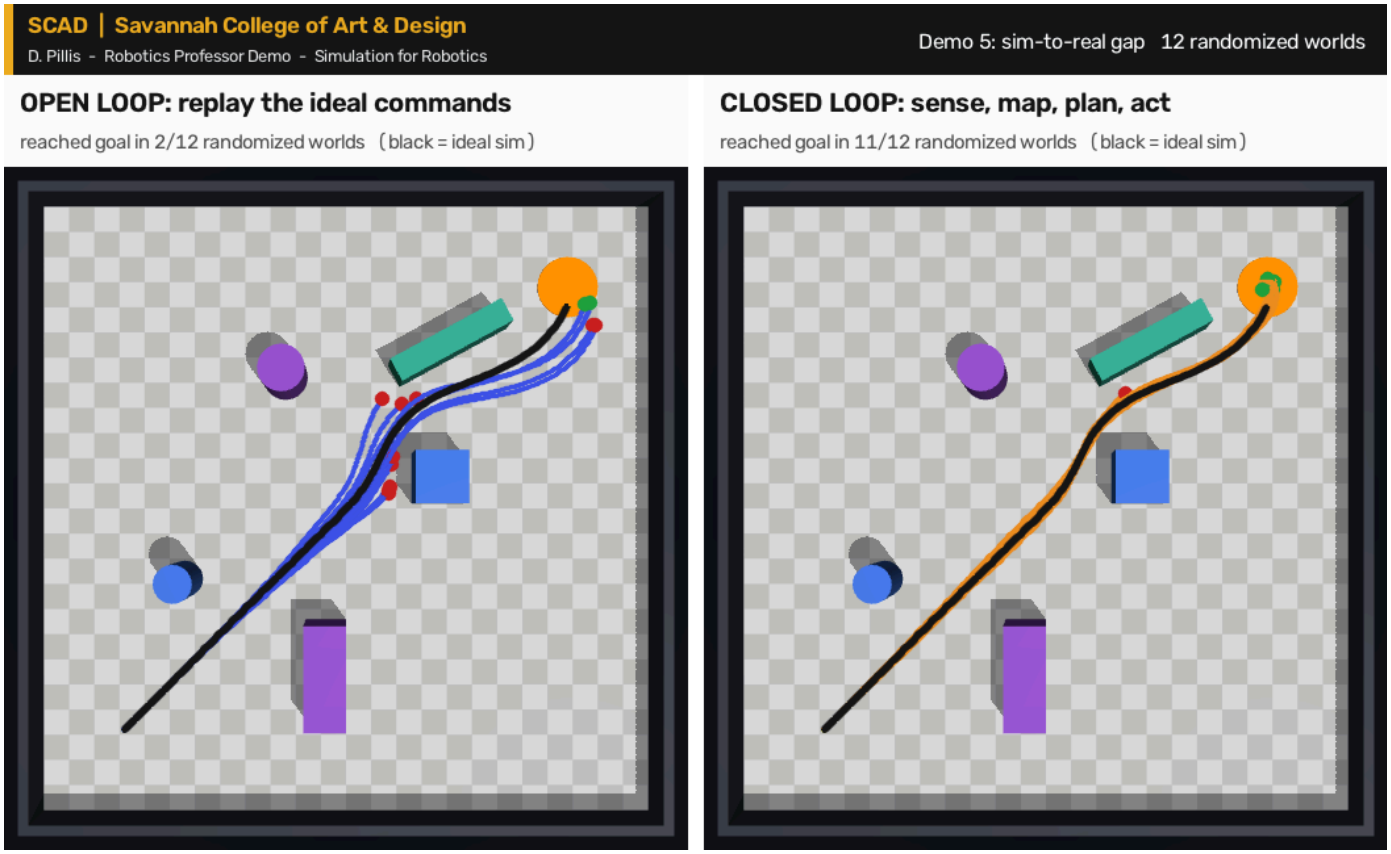


Top: studio camera with lidar rays; onboard camera. Middle: depth camera (bright = near); lidar scan; the occupancy map the robot built, with its A* plan (gold) and goal (orange). Bottom: IMU; wheel speed commanded vs. measured.

5 · The camera pipeline, frame by frame (Demo 4)



6 · The sim-to-real gap (Demo 5)



Same robot, same goal, twelve plausible "realities" (friction, mass, motor strength, sensor noise, latency). Replaying commands that were perfect in the ideal simulator fails in most of them; sensing and re-planning succeeds in nearly all.

7 · Where each platform is strongest

Presenter's summary for classroom selection (see Segment 2 for details and sources).

	Physics realism	GPU acceleration	Sensor fidelity	Runs on any student laptop	Art / design pipeline fit
Isaac Sim / Isaac Lab	●●●	●●●	●●●●	—	●●● (OpenUSD)
mjlab / MuJoCo	●●●●	●●● (Warp)	●●	●●●● (MuJoCo)	●●
Unity	●●	●●	●●●	●●●●	●●●●
Gazebo	●●●	●	●●●	●●	●
Webots	●●	●	●●	●●●●	●●

8 · Sim-to-real checklist

Before trusting a simulated result, ask:

- ☐ Did I measure the real robot's mass, wheel size and motor limits?
- ☐ Are friction values plausible, and did I test a range of them?
- ☐ Do my sensors have realistic noise, range limits, rate and latency?
- ☐ Does my controller use feedback, or does it replay a plan?
- ☐ What happens if a sensor drops out for one second?
- ☐ Could my algorithm be exploiting a simulator glitch?
- ☐ What is my first, safe hardware test?

github.com/mrpillis/scad-robotics-simulation-demo